CERTIFICATE-FIRST PROOF ORCHESTRATION: THE PROOFCTL FRAMEWORK FOR COMPUTATIONAL MATHEMATICS

ABSTRACT. We describe `proofctl`, a proof orchestration layer for computational mathematics that enforces a *certificate-first* architecture: every numerical result entering a proof must be accompanied by an independently replayable certificate, and the release gate derives claim acceptance solely from obligation results, never from self-reported outcomes. We illustrate the framework through its application to the FP-0.35 conjecture (Weil quadratic form positivity at $L = 7/20$), where 17 inter-dependent claims spanning pure rational arithmetic, Lean 4 formalisation, and 256-bit Arb interval arithmetic are orchestrated into a single offline-verifiable bundle. Key design decisions — fail-closed schema validation, identity-closure propagation, and the `scripted` runtime class — are motivated by concrete failure modes encountered during development.

1. INTRODUCTION

Computational proofs in number theory and analysis increasingly involve heterogeneous evidence: formal proofs in proof assistants, certified interval arithmetic, and independently replayable computational certificates. Orchestrating these into a single verifiable artefact requires infrastructure that is both rigorous and practical.

`proofctl` is an open-source Go tool that provides this infrastructure. Its core invariants are:

(1) **No self-reported conclusions.** A claim is accepted only when its checker emits `obligation_results` with all verdicts `"pass"`; a hand-crafted `"outcome":"accepted"` field cannot bypass the gate.
(2) **Identity closure.** Every attestation binds the exact checker digest, schema digest, and dependency digests. Changing any input automatically stalens downstream attestations.
(3) **Cold-replay requirement.** Release is blocked until every deterministic-cap claim has been independently replayed from scratch.
(4) **Offline verifiability.** The release bundle is self-contained; no network access is required to verify it.

2. ARCHITECTURE

2.1. **ProofGraph and Claims.** A *ProofGraph* is a directed acyclic graph of *claims*. Each claim has:

- a `statement_digest` (SHA-256 of the claim text),
- a list of `dependencies` with required assurance levels,
- a `checker_policy` identifying which checker verifies it,
- an `obligations` list specifying what the checker must certify.

2.2. **Checker Protocol v2.** A checker is a subprocess that receives a certificate file path and emits JSON to stdout:

Date: August 6, 2026.

```
{
  "protocol_version": 2,
  "obligation_results": [
    {"id": "thm3.log2-bounds-verified", "verdict": "pass"},
    ...
  ],
  "metadata": {"format_version": "first-prime-1.0", ...}
}
```

Exit code 0 = certified; 1 = uncertified; 2 = malformed; 3 = O2_BLOCKED.

2.3. **Release Gate.** The release gate evaluates 11 conditions against the full attestation graph. Critical conditions include:

- **C01**: Every claim in the graph is accepted.
- **C04**: Replay consistency — every deterministic-cap claim has a freshness-dated cold-replay attestation.
- **LEGACY**: No v1 attestation (protocol_version $\neq 2$) may appear in the graph.

3. Application to FP-0.35

3.1. **Proof Graph Structure.** The FP-0.35 proof graph contains 17 claims organised in four layers:

Layer	Claims	Assurance
Frozen model	1	independent-review
Analytic lemmas	8	independent-review
Deterministic-cap	7	deterministic-cap
Main conjecture	1	deterministic-cap + exact-replay

3.2. **Checker Taxonomy.**

`check_thm3.py`**:** Pure rational arithmetic: five inequalities about $\log 2$, ε, κ_{edge}, and c_2. No floating point; exit 0 iff all five hold.

`check_archimedean.py`**:** Dual-path certification: independently integrates M_K via Arb (Path A) and mpmath (Path B), verifies non-empty intersection for all 6×6 or 8×8 matrix entries, and emits leaf witnesses.

`check_fp_wrapper.py`**:** Orchestrates the full O1-B gate: calls the archimedean sub-checker, assembles F and R_η, runs mpmath LDL^T at dps=100, and emits 7 obligation results.

3.3. **Key Engineering Lessons.**

(1) **M_V + M_K not M_K alone.** The Archimedean matrix block $M^{(0)} = M_V + M_K$; using only M_K produces a matrix whose Schur complement is indefinite.

(2) **Interval blowup in LDL^T.** Running outward-rounded LDL^T on the full interval matrix amplifies off-diagonal widths; the mpmath midpoint approach with certified width bound is more robust.

(3) **Timeout asymmetry.** The archimedean sub-checker reruns full integration on each replay (≈ 600 s for the even sector); the `replay.py` timeout was raised from 600 s to 3600 s to avoid spurious O2_BLOCKED exits.

4. Comparison with Related Systems

Coq/Isabelle certificates: Formal kernels provide the strongest guarantees but require the full proof term. `proofctl` is complementary: it orchestrates *computational* evidence that formal systems could, in principle, certify.

CertiCoq / CompCert: Compiler verification uses similar certificate-passing architectures but targets a different domain.

Jupyter notebooks: Reproducible but not independently replayable in the proofctl sense: no checker digest, no obligation exact-set, no fail-closed gate.

5. Conclusion

`proofctl` demonstrates that certificate-first proof orchestration is practical for research-scale computational mathematics. The FP-0.35 case study produced a 19-member offline-verifiable bundle passing all 11 release conditions, with each checker independently replayable and each obligation explicitly attested.

References

[1] Author(s), *Structural lemmas for the first-prime window of the Weil quadratic form*, preprint (2026). doi:10.5281/zenodo.21807498.

[2] F. Johansson, *Arb: efficient arbitrary-precision midpoint-radius interval arithmetic*, IEEE Trans. Comput. **66** (2017), 1281–1292.

[3] L. de Moura et al., *The Lean 4 theorem prover and programming language*, CADE 2021.